

Design Patterns

Design Patterns

The 23 Gang of Four (GoF) design patterns, categorized into:

Creational Patterns (6)

Structural Patterns (7)

Behavioral Patterns (10)

These patterns offer solutions to common object-oriented design problems, promoting code reusability, flexibility, and maintainability.

Creational Design Patterns

1. Singleton

Definition: Ensures a class has only one instance and provides a global point of access to it. This is useful when exactly one object is needed to coordinate actions across the system. It restricts instantiation by making the constructor private and provides a static method to get the instance.

Example: Think of a president of a country—there should be only one at a time. Everyone refers to the same person for leadership.

2. Factory Method

Definition: Defines an interface for creating an object, but lets subclasses decide which class to instantiate. It allows a class to defer instantiation to its subclasses, promoting loose coupling and scalability.

Example: A restaurant menu where you choose a dish, and the kitchen prepares the appropriate item. You don't know how it's made, just that you get what you ordered.

Creational Design Patterns

3. Abstract Factory

Definition: Provides an interface for creating families of related or dependent objects without specifying their concrete classes. It ensures that products created together are compatible.

Example: A furniture factory that produces Victorian-style or Modern-style furniture sets (chair, sofa, table) that all match in design.

4. Builder

Definition: Separates the construction of a complex object from its representation so that the same construction process can create different representations. It focuses on step-by-step object creation.

Example: Building a custom burger at a fast-food joint—choose bun, patty, toppings, and sauces in any order.

Creational Design Patterns

5. Prototype

Definition: Creates new objects by copying an existing object (prototype), rather than creating from scratch. It's useful when object creation is expensive or complex.

Example: Photocopying a document instead of rewriting it from scratch.

6. Object Pool

Definition: Maintains a pool of reusable objects and provides a way to borrow and return them. It helps manage resources efficiently, especially when object creation is costly.

Example: A library where you borrow books instead of buying new ones every time.

Creational Design Patterns

Key Points to note:

- ✓ Encapsulate object creation logic.
- ✓ Promote re usability through abstraction.
- ✓ Enable scalability.
- ✓ Support object life cycle management.
- ✓ Encourage separation of concerns.
- ✓ Use lazy initialization when appropriate.
- ✓ Implement object pooling for expensive objects.
- ✓ Use builders for complex object construction.
- ✓ Ensure thread safety in shared creational logic.
- ✓ Allow for object cloning when needed.

Structural Design Patterns

7. Adapter

Definition: Converts the interface of a class into another interface that clients expect. It allows incompatible interfaces to work together.

Example: A travel plug adapter lets your charger fit into a foreign socket.

8. Bridge

Definition: Decouples an abstraction from its implementation so that the two can vary independently. It separates the interface from the implementation.

Example: A remote control (abstraction) that can work with different brands of TVs (implementations).

Structural Design Patterns

9. Composite

Definition: Composes objects into tree structures to represent part-whole hierarchies. It allows clients to treat individual objects and compositions uniformly.

Example: A folder that can contain files or other folders, and you can perform actions like open or delete on both.

10. Decorator

Definition: Adds responsibilities to objects dynamically without altering their structure. It's a flexible alternative to subclassing for extending functionality.

Example: Adding toppings to an ice cream cone—each topping wraps the original cone with extra features.

Structural Design Patterns

11. Facade

Definition: Provides a simplified interface to a complex subsystem. It hides the complexities of the system and provides a cleaner interface.

Example: A universal remote that controls your TV, sound system, and lights with one interface.

12. Flyweight

Definition: Reduces memory usage by sharing as much data as possible with similar objects. It's useful when many objects share common data.

Example: Characters in a word processor—each character doesn't store its font info separately; they share it.

Structural Design Patterns

13. Proxy

Definition: Provides a placeholder or surrogate for another object to control access to it. It can add functionality like lazy loading, access control, or logging.

Example: A receptionist who controls access to the CEO—people don't directly interact with the CEO.

Structural Design Patterns

Key Points to note:

- ✓ Prefer composition over inheritance to promote flexibility.
- ✓ Use interfaces to define contracts between components.
- ✓ Encapsulate structural changes behind abstractions.
- ✓ Apply the Adapter pattern to integrate incompatible interfaces.
- ✓ Use the Bridge pattern to separate abstraction from implementation.
- ✓ Apply the Composite pattern to treat individual and composite objects uniformly.
- ✓ Use the Decorator pattern to add responsibilities dynamically.
- ✓ Introduce a Facade to simplify complex subsystems.
- ✓ Use the Flyweight pattern to share common object data and reduce memory usage.
- ✓ Apply the Proxy pattern to control access to objects.
- ✓ Promote low coupling and high cohesion in class design.
- ✓ Design for scalability by decoupling components.

Behavioural Design Patterns

14. Chain of Responsibility

Definition: Passes a request along a chain of handlers. Each handler decides either to process the request or pass it to the next handler in the chain.

Example: Customer support escalation—your issue goes from a chatbot to an agent to a manager.

15. Command

Definition: Encapsulates a request as an object, thereby allowing for parameterization of clients with queues, logs, and undo able operations.

Example: A TV remote where each button is a command (volume up, channel down, etc.).

Behavioural Design Patterns

16. Interpreter

Definition: Defines a grammar for a language and provides an interpreter to interpret sentences in the language. It's used to interpret expressions.

Example: A calculator interpreting and evaluating mathematical expressions like $2 + 3 * 4$.

17. Iterator

Definition: Provides a way to access elements of a collection sequentially without exposing its underlying representation.

Example: A TV remote that lets you scroll through channels one by one.

Behavioural Design Patterns

18. Mediator

Definition: Defines an object that encapsulates how a set of objects interact. It promotes loose coupling by preventing objects from referring to each other explicitly.

Example: An air traffic controller who coordinates all planes instead of planes communicating directly.

19. Memento

Definition: Captures and externalizes an object's internal state so that it can be restored later without violating encapsulation.

Example: Saving a game so you can return to that exact point later.

Behavioural Design Patterns

20. Observer

Definition: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

Example: YouTube subscriptions—when a channel uploads a video, all subscribers are notified.

21. State

Definition: Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

Example: A traffic light that changes behavior (color) based on its current state.

Behavioural Design Patterns

22. Strategy

Definition: Defines a family of algorithms, encapsulates each one, and makes them interchangeable. It lets the algorithm vary independently from clients that use it.

Example: A navigation app that lets you choose between fastest, shortest, or scenic route.

23. Template Method

Definition: Defines the skeleton of an algorithm in a method, deferring some steps to subclasses. It lets subclasses redefine certain steps without changing the algorithm's structure.

Example: A coffee machine that always follows the same steps: boil water → brew → pour → add extras. The extras can vary.

Behavioural Design Patterns

Key Points to note:

- ✓ Encapsulate behavior in separate classes to promote flexibility.
- ✓ Use interfaces to define interchangeable behaviors.
- ✓ Apply the Strategy pattern to select algorithms at runtime.
- ✓ Use the Observer pattern to notify dependent objects of state changes.
- ✓ Apply the Command pattern to encapsulate requests as objects.
- ✓ Use the Iterator pattern to access elements without exposing internal structure.
- ✓ Apply the Mediator pattern to centralize complex communications.
- ✓ Use the Memento pattern to capture and restore object state.
- ✓ Apply the State pattern to change behavior based on internal state.
- ✓ Use the Template Method pattern to define the skeleton of an algorithm.
- ✓ Apply the Visitor pattern to separate algorithms from object structures.
- ✓ Use the Chain of Responsibility pattern to pass requests along a chain of handlers.
- ✓ Promote loose coupling between sender and receiver objects.
- ✓ Encourage delegation of responsibilities to improve cohesion.

THANK YOU

Nisanth Saravanan

About Relevantz

Relevantz Technology Services Inc. has been delivering relevant technology solutions for leading enterprises and ISVs for over 25+ years. Our team of 1200+ software engineers across 4 global offices serves customers across the finance, healthcare, insurance, media, telecom, retail, and technology sectors. Learn more at www.relevantz.com or [@relevantztech](https://twitter.com/relevantztech).

© 2025 Relevantz Technology Services, Inc. All rights reserved.